

Федеральное государственное автономное образовательное учреждение
высшего образования «Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Лабораторная работа №6

По дисциплине

“Вычислительная математика”

Вариант: 24

Выполнил:
Стрельбицкий Илья Павлович

Группа: Р32101

Преподаватель:
Рыбаков Степан Дмитриевич

Санкт-Петербург, 2023г

Оглавление

Цель работы	3
Описание алгоритма решения задачи	3
Метод Эйлера.....	3
Метод Рунге-Кутты 4-го порядка.....	3
Метод Адамса	4
Программная часть работы.....	4
Исходный код программы	4
Листинг программы	4
Вывод программы	5
Вывод.....	11

Цель работы

Решить задачу Коши для обыкновенных дифференциальных уравнений численными методами. В соответствии с вариантом задания нужно реализовать в программе метод Эйлера, метод Рунге-Кутты 4-го порядка и метод Адамса.

Описание алгоритма решения задачи

Метод Эйлера

Метод Эйлера основан на разложении искомой функции в ряд Тейлора в окрестностях узлов $x = x_i (i = 0, 1, \dots)$, в котором отбрасываются все члены, содержащие производные второго и более высоких порядков:

$$Y(x_i + h) = Y(x_i) + Y'(x_i) * h + O(h^2)$$

Полагаем: $Y'(x_i) = f(x_i, Y(x_i)) = f(x_i, y_i) = \frac{y_{i+1} - y_i}{h}$

Введем последовательность равноотстоящих точек (узлов) $x_0, x_1, \dots, x_n$, выбрав малый шаг $h = x_{i+1} - x_i = \text{const}$ Тогда получаем формулу Эйлера:

$$y_{i+1} = y_i + hf(x_i, y_i)$$

Метод Эйлера относится к одношаговым методам и обладает первым порядком точности $\delta_n = O(h)$.

Метод Рунге-Кутты 4-го порядка

Идея метода заключается в том, чтобы при численном решении задачи Коши не использовать в расчетных формулах частные производные функции, а использовать только ее саму, зато вычислять на каждом шаге ее значения в нескольких точках. Широко распространен метод Рунге-Кутты четвертого порядка, часто без уточнений называемый просто методом Рунге – Кутты.

$$y_{i+1} = y_i + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4)$$

$$k_1 = h * f(x_i, y_i)$$

$$k_2 = h * f(x_i + \frac{h}{2}, y_i + \frac{k_1}{2})$$

$$k_3 = h * f(x_i + \frac{h}{2}, y_i + \frac{k_2}{2})$$

$$k_4 = h * f(x_i + h, y_i + k_3)$$

Таким образом, данный метод Рунге-Кутты требует на каждом шаге четырехкратного вычисления правой части уравнения $Y' = f(x, Y)$.

Суммарная погрешность этого метода есть величина $\delta_n = O(h^4)$

Метод Рунге-Кутты требует большего объема вычислений по сравнению с методом Эйлера и его модификацией, однако это окупается повышенной точностью, что дает возможность проводить счет с большим шагом. Другими словами, для получения результатов с одинаковой точностью в методе Эйлера потребуется значительно меньший шаг, чем в методе Рунге-Кутты.

Метод Адамса

Широко распространенным семейством многошаговых методов являются методы Адамса. Простейший из них, получающийся при $k = 1$, совпадает с рассмотренным ранее методом Эйлера первого порядка точности. В практических расчетах чаще всего используется вариант метода Адамса, имеющий четвертый порядок точности и использующий на каждом шаге результаты предыдущих четырех. Именно его и называют обычно методом Адамса. Рассмотрим этот метод.

Пусть найдены значения в четырех, $y_{i-3}, y_{i-2}, y_{i-1}, y_i$, последовательных узлах ($k = 4$). При этом имеются также вычисленные ранее значения правой части $f_{i-3}, f_{i-2}, f_{i-1}, f_i$, где $f_i = f(x_i, y_i)$. В качестве интерполяционного многочлена $P_3(x)$ можно взять многочлен Ньютона. В случае постоянного шага h конечные разности для правой части в узле имеют вид:

$$\begin{aligned}\Delta f_i &= f_i - f_{i-1} \\ \Delta^2 f_i &= f_i - 2f_{i-1} + f_{i-2} \\ \Delta^3 f_i &= f_i - 3f_{i-1} + 3f_{i-2} - f_{i-3}\end{aligned}$$

Тогда разностную схему четвертого порядка метода Адамса можно записать после необходимых преобразований в виде:

$$y_{i+1} = y_i + hf_i + \frac{h^2}{2}\Delta f_i + \frac{5h^3}{12}\Delta^2 f_i + \frac{3h^4}{8}\Delta^3 f_i$$

Сравнивая метод Адамса с методом Рунге-Кутты той же точности, отмечаем его экономичность, поскольку он требует вычисления лишь одного значения правой части на каждом шаге (в методе Рунге-Кутты - четырех). Но метод Адамса неудобен тем, что невозможно начать счет по одному лишь известному значению y_0 . Расчет может быть начат только с узла x_3 , а не x_0 . Значения y_1, y_2, y_3 , необходимые для вычисления y_4 нужно получить каким-либо другим способом (например, методом Рунге-Кутты), что существенно усложняет алгоритм. Кроме того, метод Адамса не позволяет (без усложнения формул) изменить шаг h в процессе счета. Этого недостатка лишены одношаговые методы.

Программная часть работы

Исходный код программы

Ссылка на github - <https://github.com/stoneshik/compute-math-lab-sixth>

Листинг программы

Программа написана на языке Python с использованием библиотек numpy и sympy для упрощения вычислений, и также matplotlib для отрисовки графиков и prettytable для вывода таблиц в консоль.

Есть только основной модуль main.py;

В нем есть следующие классы:

Equation – класс-обёртка для функций

SolutionMethod – абстрактный класс для описания классов которые реализуют методы численные методы решения ОДУ

В нём есть следующие методы:

calc – абстрактный метод, который реализует логику вычисления
 print_solution – выводит полученный результат в виде таблицы в консоль
 draw – отрисовывает график точного решения и полученного при помощи численного метода

EulerMethod – класс реализующий логику вычисления при помощи метода Эйлера

RungeKuttaMethod – класс реализующий логику вычисления при помощи метода Рунге-Кутты 4 порядка

В нём есть следующий метод:

calc_iter – метод который вычисляет значение y в зависимости от предыдущих значений y и x методом Рунге-Кутты 4-го порядка

AdamsMethod – класс реализующий логику вычисления при помощи метода Адамса

В нём есть следующие методы:

_calc_iter – метод, который вычисляет значение y в зависимости от 4-х предыдущих значений y и x методом Адамса.

_create_rows – защищённый метод, который возвращает значения с 2 по 4 строк таблицы решений.

В модуле есть следующие функции:

input_data – функция ввода данных с клавиатуры

main – главная функция модуля

Вывод программы

Пример 1.

Выберите диф. уравнение, которое требуется решить:

1. $y' = y^2 \cdot \left(x + 1\right) + y$

2. $y' = \left(x - y\right)^2 + 1$

3. $y' = x \cdot e^{2x + 3}$

Введите номер выбранного диф. уравнения...

1

Введите начальное условие $y_0 = y(x_0)$...

-1

Задайте пределы дифференцирования $[x_0, x_n]$:

Введите значения x_0 и x_n через пробел...

1 1.5

Введите начальное значение для шага h ...

0.1

Выберите метод решения

1. метод Эйлера

2. метод Рунге-Кутты 4-го порядка

3. метод Адамса

Введите номер выбранного метода решения...

1

Введите погрешность вычислений (чтобы оставить значение по умолчанию - 0,01 нажмите Enter)...

i	x_i	y_i^h	$y_i^{h/2}$	$f(x_i, y_i)^h$	$f(x_i, y_i)^{h/2}$	R
0	1.0	-1.0	-1.0	1.0000000000000000	1.0000000000000000	0
1	1.1	-0.9000000000000000	-0.9049937500000000	0.8010000000000000	0.855885678208984	0.004993749999999991
2	1.2	1.2000000000000000	-0.8199000000000000	-0.827165140988046	0.6590192220000000	0.712289742559947

```
| 3 | 1.3000000000000003 | -0.753998077800000 | -0.762132099731912 | 0.553582055250018 |
0.602854443256255 | 0.00813402193191204 |
| 4 | 1.4000000000000004 | -0.698639872274998 | -0.706906036038750 | 0.472794538442824 |
0.517398516241894 | 0.00826616376375144 |
| 5 | 1.5000000000000004 | -0.651360418430716 | -0.659374283053442 | 0.409315568314877 |
0.449300552084773 | 0.00801386462272657 |
```

Найденное решение:

```
+-----+
| i | xi | yi |
+-----+
| 0 | 1.0 | -1.0 |
| 1 | 1.1 | -0.904993750000000 |
| 2 | 1.2000000000000002 | -0.827165140988046 |
| 3 | 1.3000000000000003 | -0.762132099731912 |
| 4 | 1.4000000000000004 | -0.706906036038750 |
| 5 | 1.5000000000000004 | -0.659374283053442 |
```

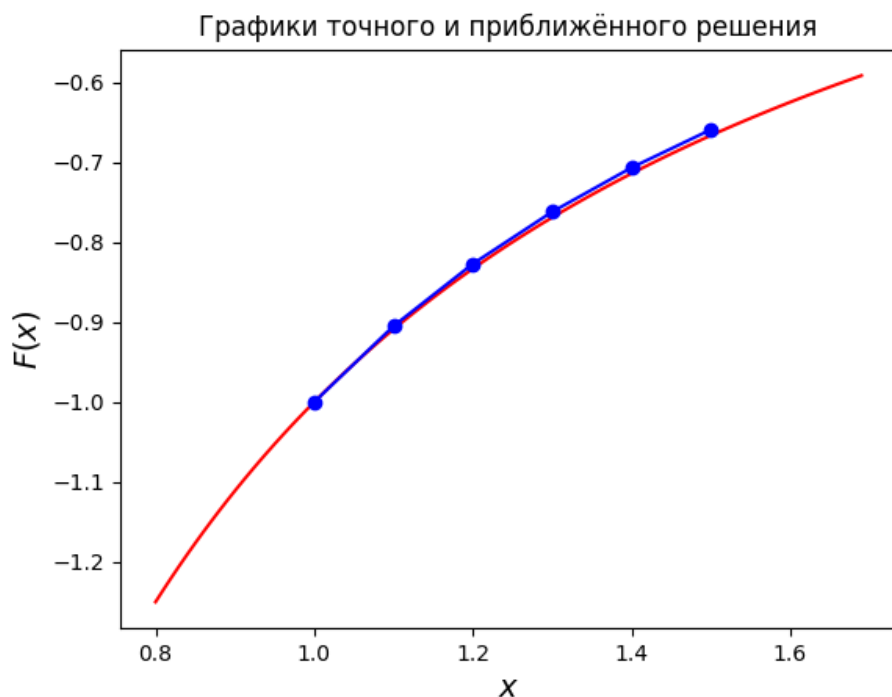


График для примера 1.

Пример 2.

Выберите диф. уравнение, которое требуется решить:

1. $y' = y^2 \left(x + 1 \right) + y$
2. $y' = \left(x - y \right)^2 + 1$
3. $y' = x e^{2x + 3}$

Введите номер выбранного диф. уравнения...

2

Введите начальное условие $y_0 = y(x_0)$...

0

Задайте пределы дифференцирования $[x_0, x_n]$:

Введите значения x_0 и x_n через пробел...

-1 -0.5

Введите начальное значение для шага h ...

0.2

Выберите метод решения

1. метод Эйлера

2. метод Рунге-Кутты 4-го порядка

3. метод Адамса

Введите номер выбранного метода решения...

2

Введите погрешность вычислений (чтобы оставить значение по умолчанию - 0,01 нажмите Enter)...

0.001

i	x_i	y_i^h	$y_i^{h/2}$	R
0	-1.0	0.0	0.0	0
1	-0.8	0.449970142137175	0.449997992047015	1.85666065604106e-6
2	-0.6000000000000001	1.06647306853048	1.06665325725032	1.20125813227290e-5
3	-0.4000000000000001	2.09842852921784	2.09988288414075	9.69569948608454e-5

Найденное решение:

i	x_i	y_i
0	-1.0	0.0
1	-0.8	0.449997992047015
2	-0.6000000000000001	1.06665325725032
3	-0.4000000000000001	2.09988288414075

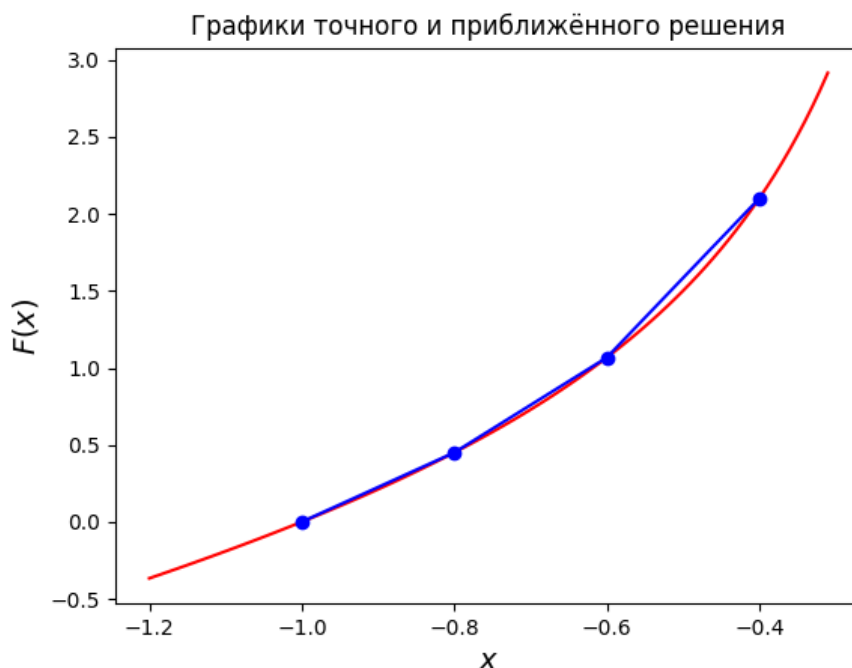


График для примера 2.

Пример 3.

Выберите диф. уравнение, которое требуется решить:

1. $y' = y^2 \left(x + 1 \right) + y$

2. $y' = \left(x - y \right)^2 + 1$

3. $y' = x e^{2x + 3}$

Введите номер выбранного диф. уравнения...

3

Введите начальное условие $y_0 = y(x_0)$...

-5

Задайте пределы дифференцирования $[x_0, x_n]$:

Введите значения x_0 и x_n через пробел...

Внимание! Введённое начальное условие для y_0 сильно отличается от точного значения, поэтому может быть получено неточное решение, либо решение не будет найдено

Введите начальное значение для шага h ...

Выберите метод решения

1. метод Эйлера
2. метод Рунге-Кутты 4-го порядка
3. метод Адамса

Введите номер выбранного метода решения...

3

Введите погрешность вычислений (чтобы оставить значение по умолчанию - 0,01 нажмите Enter)...

0.1

i	xi	yi	Точное значение	R
0	0.0	-5.0	-5.0	0
1	0.00625	-4.99960441984920	-5.02098865064827	0.0213842307990628
2	0.0125	-4.99840441755279	-5.01978864835403	0.0213842308012406
3	0.018750000000000003	-4.99637981892849	-5.01776404973194	0.0213842308034540
4	0.025	-4.99393349355891	-5.01489430035339	0.0209608067944833
5	0.03125	-4.99063154704172	-5.01115845916953	0.0205269121278162
6	0.037500000000000006	-4.98645284164997	-5.00653519203716	0.0200823503871943
7	0.043750000000000004	-4.98137584327117	-5.00100276514294	0.0196269218717724
8	0.05	-4.97537861477979	-4.99453903832437	0.0191604235445801
9	0.05625	-4.96843880930577	-4.98712145828593	0.0186826489801621
10	0.0625	-4.96053366339748	-4.97872705170897	0.0181933883114942
11	0.06875	-4.95163999007744	-4.96933241825347	0.0176924281760273
12	0.075	-4.94173417178934	-4.95891372345033	0.0171795516609814
13	0.081249999999999999	-4.93079215323458	-4.94744669148236	0.0166545382477858
14	0.0875	-4.91878943409669	-4.93490659785237	0.0161171637556770
15	0.09375	-4.90570106165213	-4.92126826193661	0.0155672002844760
16	0.1	-4.89150162326540	-4.90650603942187	0.0150044161564677
17	0.106250000000000001	-4.87616523876711	-4.89059381462456	0.0144285758574476
18	0.112500000000000002	-4.85966555271295	-4.87350499268981	0.0138394399768540
19	0.118750000000000002	-4.84197572652197	-4.85521249166899	0.0132367651470107
20	0.125000000000000003	-4.82306843049224	-4.83568873447370	0.0126203039814561
21	0.131250000000000003	-4.80291583569209	-4.81490564070444	0.0119898050123464
22	0.137500000000000004	-4.78148960572508	-4.79283461835201	0.0113450126269283
23	0.143750000000000004	-4.75876088836677	-4.76944655536982	0.0106856670030506
24	0.150000000000000002	-4.73470030707142	-4.74471181111513	0.0100115040437050
25	0.156250000000000003	-4.70927795234661	-4.71860020765720	0.00932225531059849
26	0.162500000000000003	-4.68246337299379	-4.69108102095051	0.00861764795672038
27	0.168750000000000004	-4.65422556721289	-4.66212297187080	0.00789740465790789
28	0.175000000000000004	-4.62453297356879	-4.63169421711218	0.00716124354338898
29	0.181250000000000005	-4.59335346181765	-4.59976233994291	0.00640887812526625
30	0.187500000000000006	-4.56065432359099	-4.56629434081796	0.00564001722697149
31	0.193750000000000006	-4.52640226293548	-4.53125662784613	0.00485436491064739
32	0.200000000000000007	-4.49056338670613	-4.49461500710955	0.00405162040342066
33	0.206250000000000007	-4.45310319481081	-4.45633467283341	0.00323147802259971
34	0.212500000000000008	-4.41398657030379	-4.41638019740354	0.00239362709975044
35	0.218750000000000008	-4.37317776932620	-4.37471552122983	0.00153775190362637
36	0.225000000000000001	-4.33064041089087	-4.33130394245284	0.000663531561966302
37	0.231250000000000001	-4.28633746650953	-4.28610810649163	0.000229360017896418
38	0.237500000000000001	-4.24023124965974	-4.23908999543015	0.00114125422958633
39	0.243750000000000001	-4.19228340508936	-4.19021091723991	0.00207248784944358
40	0.250000000000000001	-4.14245489795605	-4.13943149483654	0.00302340311951710
41	0.256250000000000014	-4.09070600279938	-4.08671165496761	0.00399434783176922
42	0.262500000000000001	-4.03699629234295	-4.03201061692940	0.00498567541355399
43	0.268750000000000016	-3.98128462612418	-3.97528688110982	0.00599774501435668
44	0.275000000000000013	-3.92352913894905	-3.91649821735523	0.00703092159382290
45	0.281250000000000001	-3.86368722916922	-3.85560165315812	0.00808557601110316

46	0.2875000000000001	-3.80171554677896	-3.79255346166344	0.00916208511552519
47	0.29375000000000007	-3.73756998132907	-3.72730914949046	0.0102608318386106
48	0.30000000000000004	-3.67120564965526	-3.65982344436780	0.0113822052874619
49	0.30625	-3.60257688341802	-3.59005028257849	0.0125266008395259
50	0.3125	-3.53163721645137	-3.51794279621259	0.0136944202387812
51	0.31875	-3.45833937191757	-3.44345330022423	0.0148860716933434
52	0.32499999999999996	-3.38263524926483	-3.36653327929031	0.0161019699745215
53	0.33124999999999993	-3.30447591098528	-3.28713337446795	0.0173425365173348
54	0.3374999999999999	-3.22381156917008	-3.20520336964754	0.0186081995225313
55	0.3437499999999999	-3.14059157185871	-3.12069217779859	0.0198993940601184
56	0.34999999999999987	-3.05476438917947	-3.03354782700506	0.0212165621744189
57	0.35624999999999984	-2.96627759927795	-2.94371744628727	0.0225601529906840
58	0.3624999999999998	-2.87507787403041	-2.85114725120712	0.0239306228232943
59	0.3687499999999998	-2.78111096453894	-2.75578252925338	0.0253284352855569
60	0.3749999999999998	-2.68432168640505	-2.65756762500393	0.0267540614011241
61	0.38124999999999976	-2.58465390477851	-2.55644592506145	0.0282079797170609
62	0.38749999999999973	-2.48205051917804	-2.45235984275945	0.0296906764185905
63	0.3937499999999997	-2.37645344808057	-2.34525080263503	0.0312026454455325
64	0.3999999999999997	-2.26780361327551	-2.23505922466505	0.0327443886104608
65	0.40624999999999967	-2.15604092398074	-2.12172450826213	0.0343164157186040
66	0.41249999999999964	-2.04110426071662	-2.00518501602710	0.0359192446895231
67	0.4187499999999996	-1.92293145893456	-1.88537805725397	0.0375534016805870
68	0.4249999999999996	-1.80145929239649	-1.76223987118423	0.0392194212122612
69	0.4312499999999996	-1.67662345630157	-1.63570561000632	0.0409178462952497
70	0.43749999999999956	-1.54835855015636	-1.50570932159685	0.0426492285595155
71	0.44374999999999953	-1.41659806038475	-1.37218393199955	0.0444141283852002
72	0.4499999999999995	-1.28127434267374	-1.23506122763827	0.0462131150354741
73	0.4562499999999995	-1.14231860405119	-1.09427183725985	0.0480467667913405
74	0.46249999999999947	-0.999660884691670	-0.949745213603237	0.0499156710884328
75	0.46874999999999944	-0.853230039446237	-0.801409614790415	0.0518204246558223
76	0.4749999999999994	-0.702953719092275	-0.649192085435407	0.0537616336568681
77	0.4812499999999994	-0.548758351299111	-0.493018437466969	0.0557399138321418
78	0.4874999999999994	-0.390569121305319	-0.332813230660864	0.0577558906444556
79	0.49374999999999936	-0.228309952303439	-0.168499752877416	0.0598101994260232
80	0.49999999999999933	-0.0619034855278015	-1.81848369806206e-14	0.0619034855277833
81	0.5062499999999993	0.108728939958888	0.172765344429807	0.0640364044709192

Найденное решение:

i	xi	yi
0	0.0	-5.0
1	0.00625	-4.99960441984920
2	0.0125	-4.99840441755279
3	0.01875000000000003	-4.99637981892849
4	0.025	-4.99393349355891
5	0.03125	-4.99063154704172
6	0.03750000000000006	-4.98645284164997
7	0.04375000000000004	-4.98137584327117
8	0.05	-4.97537861477979
9	0.05625	-4.96843880930577
10	0.0625	-4.96053366339748
11	0.06875	-4.95163999007744
12	0.075	-4.94173417178934
13	0.08124999999999999	-4.93079215323458
14	0.0875	-4.91878943409669
15	0.09375	-4.90570106165213
16	0.1	-4.89150162326540
17	0.10625000000000001	-4.87616523876711
18	0.11250000000000002	-4.85966555271295
19	0.11875000000000002	-4.84197572652197
20	0.12500000000000003	-4.82306843049224

21	0.13125000000000003	-4.80291583569209
22	0.13750000000000004	-4.78148960572508
23	0.14375000000000004	-4.75876088836677
24	0.15000000000000002	-4.73470030707142
25	0.15625000000000003	-4.70927795234661
26	0.16250000000000003	-4.68246337299379
27	0.16875000000000004	-4.65422556721289
28	0.17500000000000004	-4.62453297356879
29	0.18125000000000005	-4.59335346181765
30	0.18750000000000006	-4.56065432359099
31	0.19375000000000006	-4.52640226293548
32	0.20000000000000007	-4.49056338670613
33	0.20625000000000007	-4.45310319481081
34	0.21250000000000008	-4.41398657030379
35	0.21875000000000008	-4.37317776932620
36	0.22500000000000001	-4.33064041089087
37	0.23125000000000001	-4.28633746650953
38	0.23750000000000001	-4.24023124965974
39	0.24375000000000001	-4.19228340508936
40	0.25000000000000001	-4.14245489795605
41	0.25625000000000014	-4.09070600279938
42	0.26250000000000001	-4.03699629234295
43	0.26875000000000016	-3.98128462612418
44	0.27500000000000013	-3.92352913894905
45	0.28125000000000001	-3.86368722916922
46	0.28750000000000001	-3.80171554677896
47	0.29375000000000007	-3.73756998132907
48	0.30000000000000004	-3.67120564965526
49	0.30625	-3.60257688341802
50	0.3125	-3.53163721645137
51	0.31875	-3.45833937191757
52	0.32499999999999996	-3.38263524926483
53	0.33124999999999993	-3.30447591098528
54	0.33749999999999999	-3.22381156917008
55	0.34374999999999999	-3.14059157185871
56	0.34999999999999987	-3.05476438917947
57	0.35624999999999984	-2.96627759927795
58	0.36249999999999998	-2.87507787403041
59	0.36874999999999998	-2.78111096453894
60	0.37499999999999998	-2.68432168640505
61	0.38124999999999976	-2.58465390477851
62	0.38749999999999973	-2.48205051917804
63	0.39374999999999997	-2.37645344808057
64	0.39999999999999997	-2.26780361327551
65	0.40624999999999967	-2.15604092398074
66	0.41249999999999964	-2.04110426071662
67	0.41874999999999996	-1.92293145893456
68	0.42499999999999996	-1.80145929239649
69	0.43124999999999996	-1.67662345630157
70	0.43749999999999956	-1.54835855015636
71	0.44374999999999953	-1.41659806038475
72	0.44999999999999995	-1.28127434267374
73	0.45624999999999995	-1.14231860405119
74	0.46249999999999947	-0.999660884691670
75	0.46874999999999944	-0.853230039446237
76	0.47499999999999994	-0.702953719092275
77	0.48124999999999994	-0.548758351299111
78	0.48749999999999994	-0.390569121305319
79	0.49374999999999936	-0.228309952303439
80	0.49999999999999933	-0.0619034855278015
81	0.50624999999999993	0.108728939958888

+-----+-----+

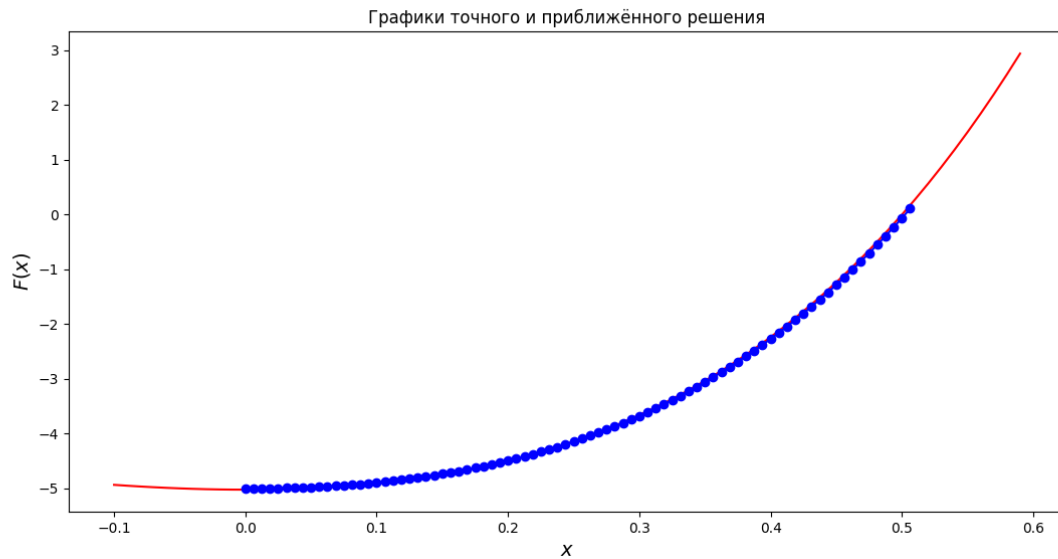


График для примера 3.

Вывод

Во время выполнения лабораторной работы были изучены одношаговые и многошаговые методы для решения задачи Коши. Также были программно реализованы на языке Python методы Эйлера, Рунге-Кутты 4-го порядка и Адамса. Точность этих методов зависит от их порядка – метод Эйлера обладает первым порядком точности, методы Рунге-Кутты и Адамса 4-м порядком точности. Метод Рунге-Кутты обладает примерно такой же точностью, что и метод Адамса, но требует больше вычислений, в то время как метод Адамса требует меньше вычислений, но для его применения требуется знать значения как минимум 4-х узлов. Метод Эйлера обладает низкой точностью, но является крайне простым в своей реализации и требует малое количество вычислений.